

Exercise Sheet No. 6

Issued: 2022-11-25
Due: 2022-12-02
1600 hrs (4 pm)

Data Structures and Decomposition

Note: Submitting your work will not earn you any bonus points. Your tutor will provide you with brief feedback on your submission. This exercise is therefore intended as practice and a way for you to check your learning progress!

You may use anything you like for this sheet, but simply copying and uploading automatically generated solutions (e.g. via an LLM) is not helpful. The sources must be cited!

At the beginning of all your source code files (.py), specify the variable `__author__` (at the beginning of the source code) – this is good practice, as it will be important for the code review tasks that are relevant to your coursework: `__author__ = "<Matr-Nr>, <Nachname>"`

Digital submissions that are not in .pdf or .txt format for texts or .py for code will not be reviewed. If submitting multiple files, they must be combined into a .zip file.

Whenever we refer to Python programmes, we mean Python code that is executable with at least Python version 3.10. Please work completely without imports at this time.

Please check the rules document in the Moodle course for further details on bonuspoints: <https://moodle.studiumdigitale.uni-frankfurt.de/moodle/mod/resource/view.php?id=323922>

(currently only in German, but we are sure you can translate it with online tools if needed).

Exercise 1: More “Fun” with Lists

Implement the following Python 3 programs. No separate documentation and no README is required.

- a) Write a Python 3 program that reads in a comma-delimited sequence of the words *zero, one, two, three, four, five, six, seven, eight, nine* from the console and converts it to a decimal number. For example:

`one,two,three` → 123

- b) Given the list `a = [2, 4, 6, 8, 10, 12, 14, 16]`, write a Python 3 program that reads in a number from the console and inserts it into the list `a` such that the sorting of `a` is maintained. Output the index at which the new element is inserted, as well as the resulting list, to the console.

- c) Given the following list of tuples:

`c = [(35, 59), (75, 60), (48, 25), (21, 4), (2, 100), (31, 6)]`

Write a Python 3 program that reads in a number from the console and searches for a matching tuple based on the first entry. If such a tuple is found, the second entry should be output, otherwise an appropriate error message. (For example, for the input 35, the output should be 59.)

Exercise 2: Iteration, Recursion

(1 pt. each) For this task, no documentation and no test cases are required.

- a) Implement the Ackermann function

$$ack(n, m) = \begin{cases} m + 1 & n = 0 \\ ack(n - 1, 1) & m = 0 \wedge n \neq 0 \\ ack(n - 1, ack(n, m - 1)) & \text{otherwise} \end{cases}$$

as a recursive Python function.

- b) Write an iterative implementation of the following recursively-defined function:

$$f(n) = \begin{cases} 1 & n \leq 1 \\ f(n - 1) + f(n - 3) & \text{otherwise} \end{cases}$$

- c) What is the inherent risk of recursive programming that doesn't exist when writing iterative code? (At least in languages like Python, C++ or Java that are not optimized for recursion.)

Exercise 3: Programming with Dictionaries and Functions

- a) Write a function `create_matrix(n, m)` which creates an $n \times m$ matrix of random integers and saves them as a dictionary. The dictionary should store the individual numbers as its values.

Reminder: Any immutable data types can be used as dictionary keys.

The matrix should be filled with random integers in the range from -2000 to 4000 (both inclusive), and should then be *returned* by the function.

The values for n and m are to be passed as function parameters and may not take values less than 2. Invalid parameters should produce a meaningful error message.

- b) Let the user enter the values for n and m , call the `create_matrix` function, and output the resulting matrix row by row. For each row, print an opening parenthesis, then print the numbers in that row separated by spaces, and then print a closing parenthesis. For example:

```
(1 2 -3)
(56 -772 320)
```

- c) Write a function `format_matrix(mtrx, n, m)` that receives a matrix dictionary from task a) as well as information about the dimensions of the matrix, and outputs the matrix so that the numbers in each column are right-justified. E.g.:

```
( 1    2  -3)
(56 -772 320)
```

Extend your program from task b) so that `format_matrix` is called in addition to the output from task b). A complete interaction with the program should look something like this:

```
Number of rows: 2
Number of columns: 3

(-1199 2732 1064)
(2016 -81 -8)

(-1199 2732 1064)
( 2016 -81 -8)
```

d) Write a short documentation for your implementation.